

MAD 2 PROJECT REPORT

Personal Details:

Name : **Sanaya Ranjit**

Roll no. : **23f3002117**

Student Email : 23f3002117@ds.study.iitm.ac.in

I am deeply fascinated by the process of turning machine learning models into functional, end-to-end applications. Rather than just training models in isolation, I enjoy building data pipelines, handling the core ML engineering, and packaging everything into a clean web application from scratch.

My focus is on mastering both data workflows and web development to build reliable, data-driven products.

I am currently pursuing the BS degree as standalone, I find extreme joy in coding, especially the rush of getting things right and searching and fixing mistakes on my own. I am a budding developer, trying to venture into the world of tech but the journey has been fascinating nonetheless.

Project Details:

Trekking Management Application

Project Statement: to build a **Trekking Management Application (TMA)** web application that allows **Admin, Trek Staff, and Users (Trekkers)** to interact with the system based on their roles.

AI/LLM usage :

I utilized AI as a pair-programming assistant during this project. I used it primarily to help generate standard Bootstrap 5 UI layouts, debug Vue.js reactivity errors, and understand the configuration syntax.

The extent of AI/LLM usage is around 25–30%, limited to help in debugging, error analysis and ui enhancing. All business logic, database architecture, and final implementations were reviewed, understood, and integrated by me.

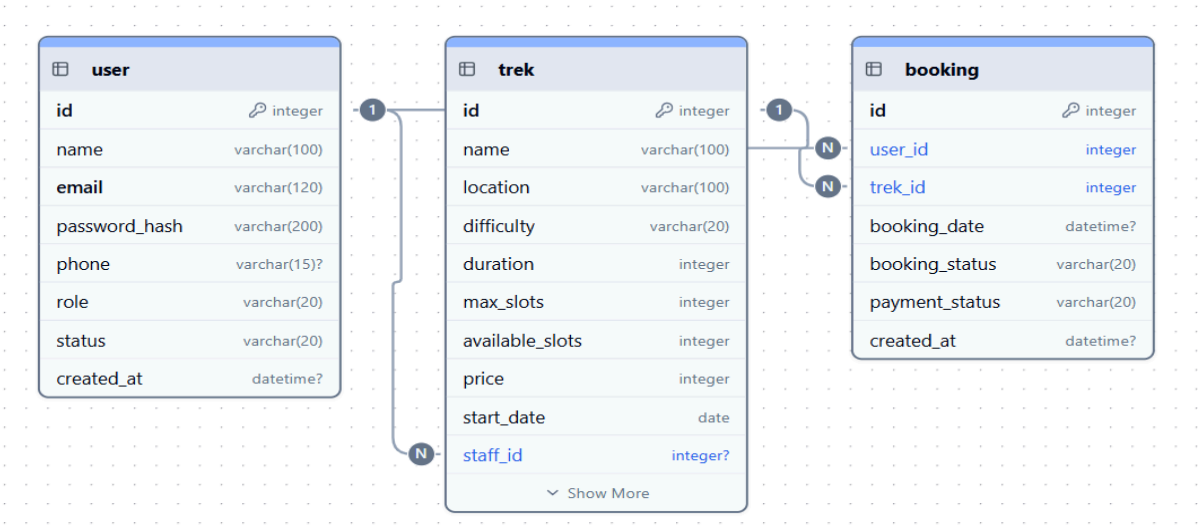
Description:

The application follows a Single Page Application (SPA) architecture with Flask as the backend API server and Vue 3 (via CDN) as the frontend framework. Session-based authentication is used for role-based access control. Redis provides caching for frequently accessed data, and Celery handles background tasks like report generation and CSV export.

Technologies and Frameworks used:

<u>Technology</u>	<u>Purpose</u>
Flask	Backend web framework and API server
Flask-sqlalchemy	Object Relational Mapper for SQLite
Werkzeug	Password hashing and security utilities
Vue 3 (CDN)	Frontend reactive UI framework
Bootstrap 5 (CDN)	CSS framework for layout and components
Bootstrap Icons (CDN)	Icon Library
Chart.js(CDN)	Data visualization for dashboard charts
Redis	In-memory caching for API performance
Celery	Distributed task queue for background jobs
Sqlite	Lightweight file-based database

ER Diagram:



API Endpoints:

Endpoint	Method	Description
<code>/login</code>	POST	Authenticates the user (Admin, Staff, or Trekker) and creates a session.
<code>/register</code>	POST	Registers a new Trekker account and saves it to the database.
<code>/treks</code>	GET	Fetches the list of all available treks to display on the dashboard.
<code>/bookings</code>	POST	Creates a new trek booking for the currently logged-in user.
<code>/bookings</code>	GET	Retrieves the active and completed booking history for the logged-in user.
<code>/stats/admin</code>	GET	Fetches dynamic statistics (total treks, user counts) for the Admin Chart.js dashboard.
<code>/export-csv</code>	POST	Triggers the asynchronous Celery task to generate a CSV file of the user's trekking history.

Architecture:

- Backend: Flask serves both the HTML entry point and JSON API endpoints
 - Frontend: Vue 3 SPA with component-based architecture
 - Database: SQLite for persistence, Redis for caching
 - Background Tasks: Celery workers for async operations
-

Features:

- User registration and login with session-based authentication
- Role-based access control (Admin, Staff, User)
- Admin: Create, update, delete treks; manage staff and users
- Staff: View assigned treks, update slots and status, view participants
- User: Browse treks with filters, book treks, view history, update profile
- Redis caching for trek listings with TTL expiry
- Celery background tasks: daily reminders, monthly reports, CSV export • Prevent overbooking and duplicate bookings

Additional Features

- Dashboard charts using Chart.js
- Search and filter functionality
- Responsive design using Bootstrap grid
- Frontend and backend validation

Video :

<https://drive.google.com/file/d/1L6Pm7qvl3VMojkD53tFnimQoU7q9hdkF/view?usp=sharing>
